Rust Workshop: Part 1

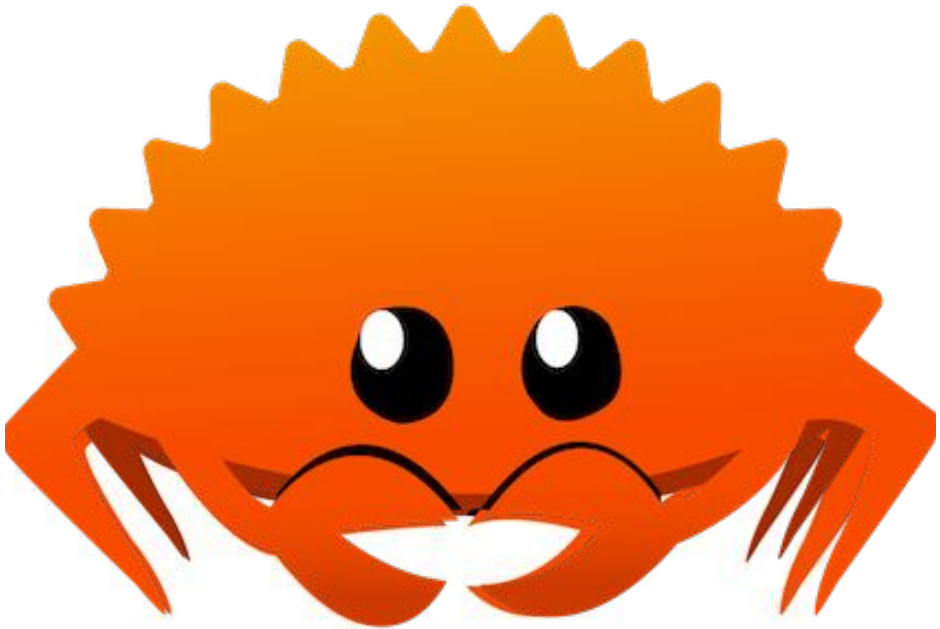


What is the Rust Programming?

Rust is a memory safe, systems level programming language.

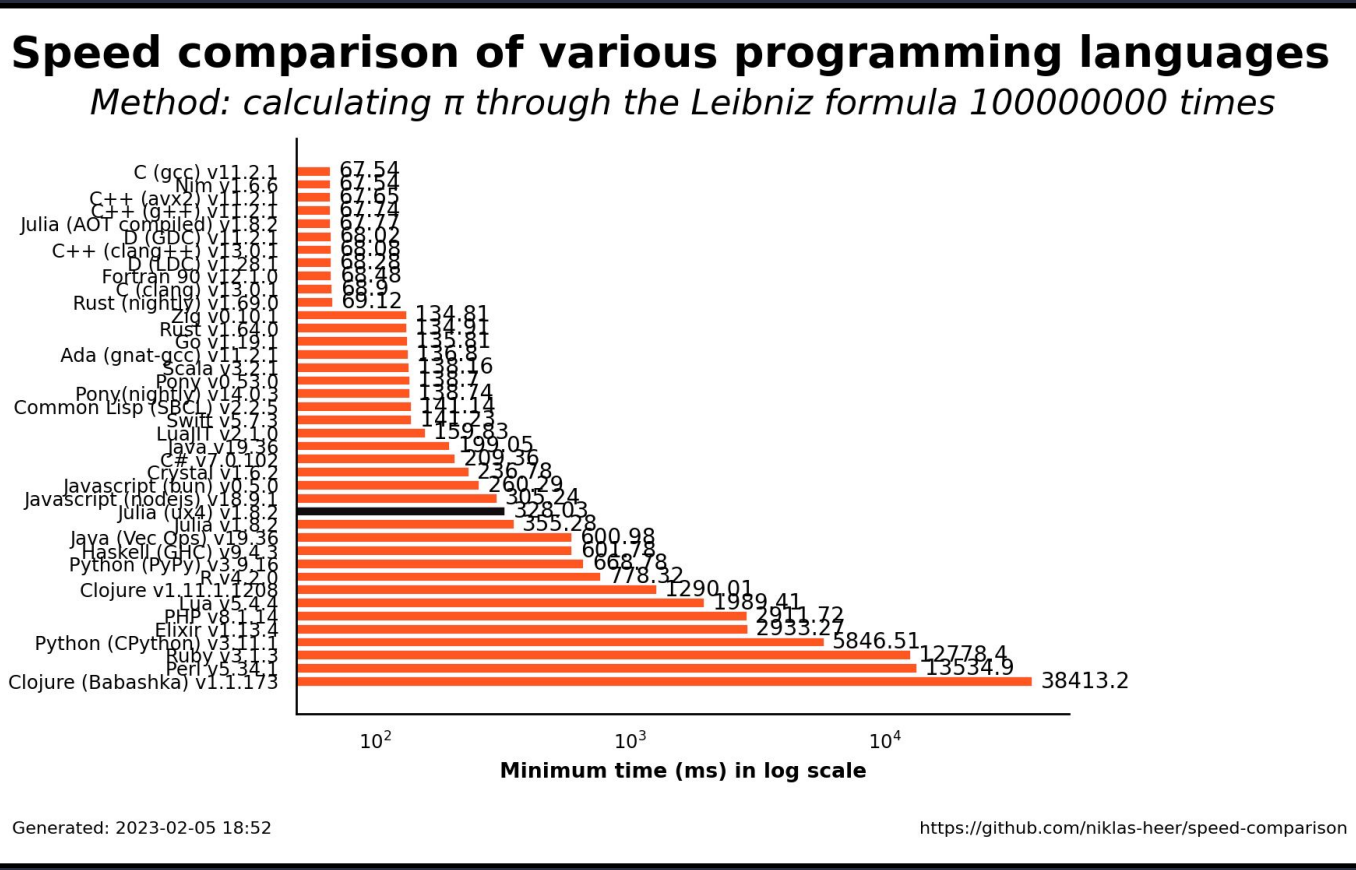
Extensive tooling for developing:

- Embedded systems
- Networks applications
- Asynchronous systems
- AND MORE!!!



Why Use Rust?

Rust is FAST


Speed comparison of various programming languages
Method: calculating π through the Leibniz formula 100000000 times
C (gcc) v11.2.1 67.54
Nim v1.6.6 67.54
C++ (avx2) v11.2.1 67.65
C++ (g++) v11.2.1 67.74
Julia (AOT compiled) v1.8.2 67.77
D (GDC) v12.2.1 68.02
C++ (clang++) v13.0.1 68.08
D (LDC) v1.28.1 68.28
Fortran 90 v12.1.0 68.48
C (clang) v13.0.1 68.9
Rust (nightly) v1.69.0 69.12
Zig v0.10.1 134.81
Rust v1.64.0 134.91
Go v1.19.1 135.81
Ada (gnat-gcc) v11.2.1 136.8
Scala v3.2.1 138.16
Pony v0.53.0 138.74
Pony(nightly) v14.0.3 138.74
Common Lisp (SBCL) v2.2.5 141.14
Swift v5.7.3 141.73
LuaJIT v2.1.0 159.83
Java v19.36 199.05
C# v7.0.102 209.36
Crystal v1.6.2 236.78
Javascript (bun) v0.5.0 260.29
Javascript (nodejs) v18.9.1 305.24
Julia (ux4) v1.8.2 328.03
Julia v1.8.2 355.28
Java (Vec Ops) v19.36 600.98
Haskell (GHC) v9.4.3 601.78
Python (PyPy) v3.9.16 668.78
R v4.2.0 778.32
Clojure v1.11.1.1208 1290.01
Lua v5.4.4 1989.41
PHP v8.1.14 2911.72
Elixir v1.13.4 2933.27
Python (CPython) v3.11.1 5846.51
Ruby v3.1.3 12778.4
Perl v5.34.1 13534.9
Clojure (Babashka) v1.1.173 38413.2
10^2
10^3
10^4
Minimum time (ms) in log scale
Generated: 2023-02-05 18:52
https://github.com/niklas-heer/speed-comparison

Rust is SAFE

Compile Time Memory Safety Mechanisms Including:

- Borrow Checker
- Immutability by default
- Ownership



And Lastly...

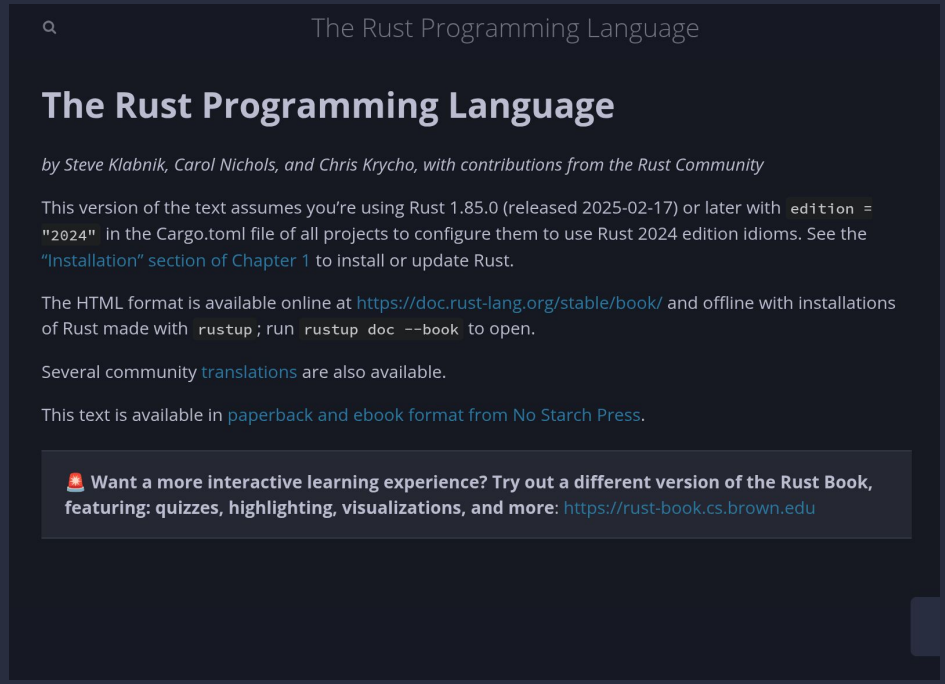
DOCUMENTATION!!!

Rust has extensive documentation available online.

Rust Programming Book available online for FREE!!!

[Rust Book Link]





Getting Started

Installation

Follow the link located here to begin installing on your machine locally.

Follow the instructions and you can begin developing with Rust.

Otherwise, you can use the Rust Playground

For Part 2, you will need to install Rust locally.

Rust Install



Rust Playground



Get this workshop's code:

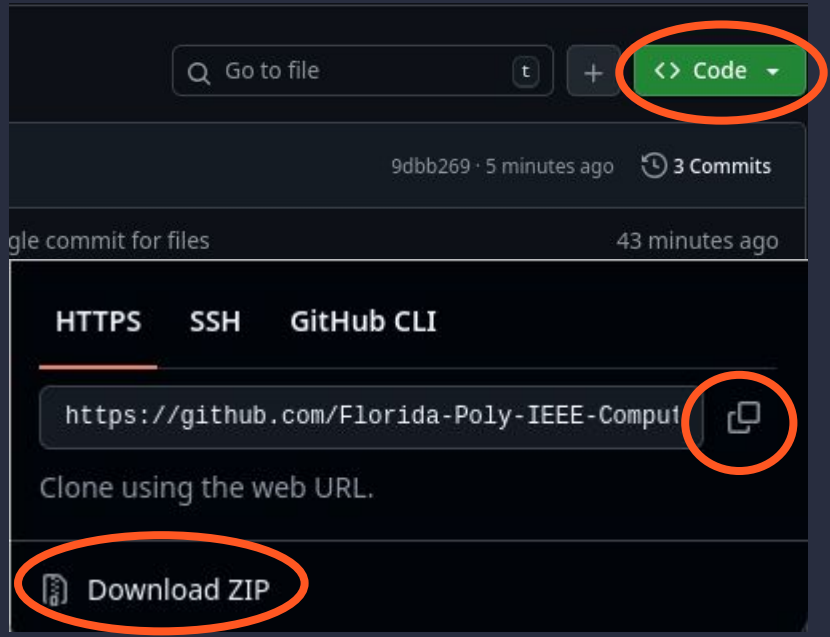
Use the link below or scan the qr code here to access the code for this workshop:

Source code ->



Get the Repository

- Highly suggested that you have git installed on your machine to go between the different branches.
- Rust Workshop Repository
- Either download the zip or run the command “git clone <repository url>.



Your First Rust Project

Open in your terminal application and run “cargo new hello-world”

Enter that directory using “cd hello-world”

Run “cargo run”

In rust playground, write the code to the right.

```
(base) noahcampise@Noahs-MBP-2 project % cargo new hello-world
    Creating binary (application) `hello-world` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
(base) noahcampise@Noahs-MBP-2 project % cd hello-world
(base) noahcampise@Noahs-MBP-2 hello-world % cargo run
   Compiling hello-world v0.1.0 (/Users/noahcampise/project/hello-world)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.69s
     Running `target/debug/hello-world`
Hello, world!
```

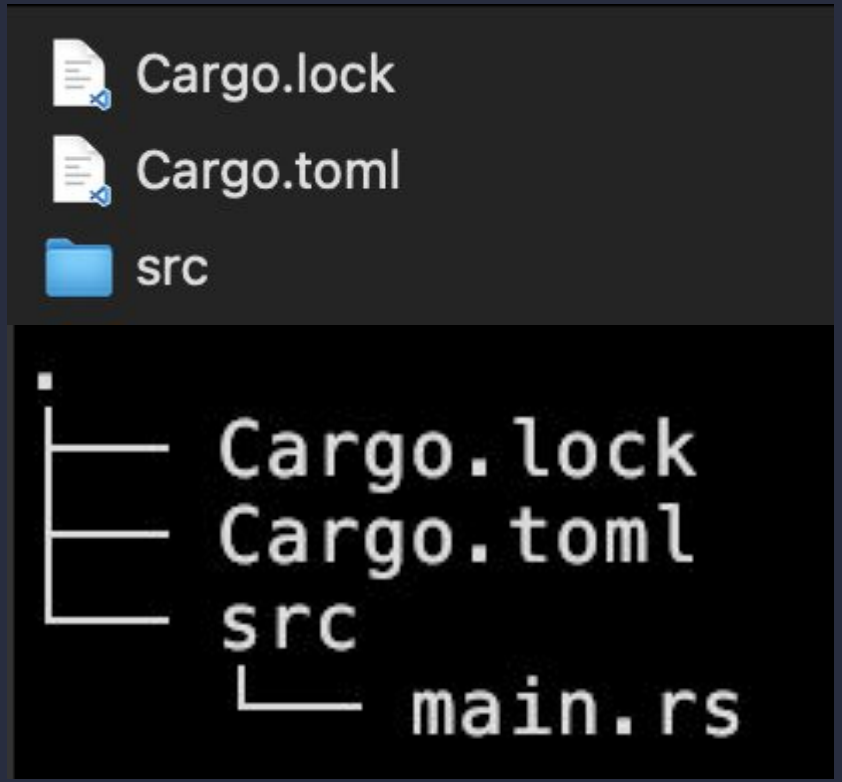
File Breakdown

To the right is the file breakdown of a Rust Project.

src/main.rs -> Your Code

Cargo.toml -> Project Configurations (do not worry for this workshop)

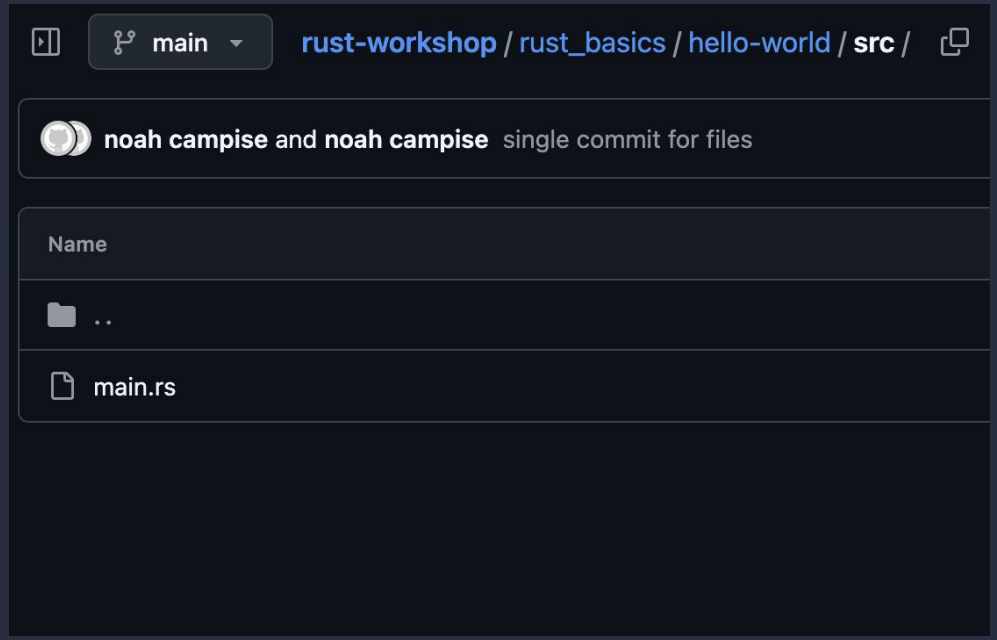
Cargo.lock -> Auto Generated file



How to use the slides with the source code.

Within the GitHub Repository, there are the directories/folders that hold the source code.

For a given slide, check the bottom left corner for the path to the source code shown in the slide.

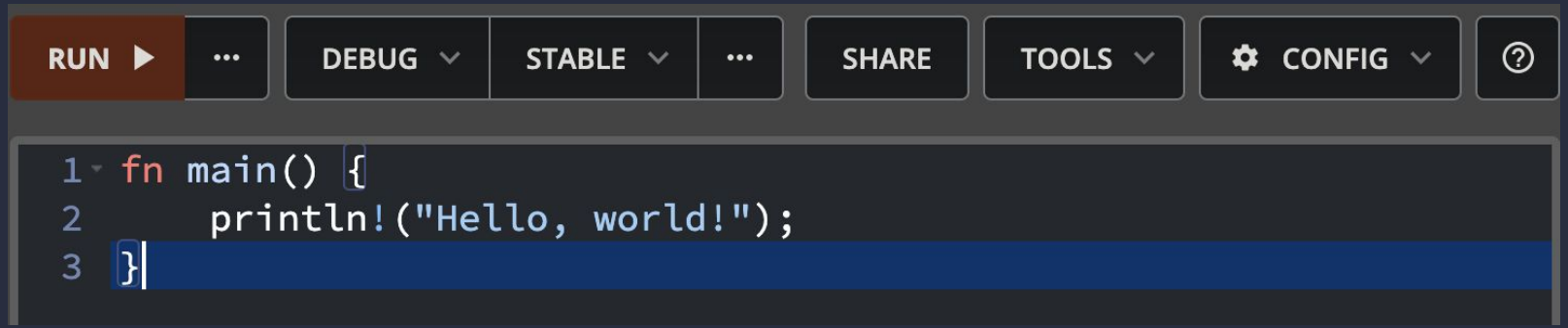


rust-workshop/hello-world

Rust Playground

If you were unable to install Rust, you can use Rust Playground instead -> click-here

We will now only cover code :)



Terminology

- Code Block: Any amount of code that is wrapped in curly brackets {}
- Variable: A named component that holds a value
- Function: A named code block that can be reran by calling the function name followed by parentheses.
- Macro: A meta Function (we will not be making macros)

Breakdown of hello-world

Below is the code for the hello-world program that we created earlier.

Let's break it down.

```
// main is the entry point for your code
fn main() {
    // This is a macro that would be used to print to console
    // If it were a function, you would not use a !
    // It would be println(input);
    println!("Hello, world!");
}
```

rust-workshop/hello-world

Guessing Game

Let's make a "Guessing Game"

Below is the code to create a guessing game. It will make a guess of what you typed.

It will ALWAYS be right LOL.

Code for Guessing Game

```rust
use std::io; // to get user input, we need to bring std::io into scope

fn main() {
    println!("Guess the number!");

    println!("Please input your guess.");

    let mut guess = String::new();

    io::stdin().read_line(&mut guess).expect("Failed to read line");

    println!("You guessed: {guess}");
}
```

rust-workshop/guessing-game

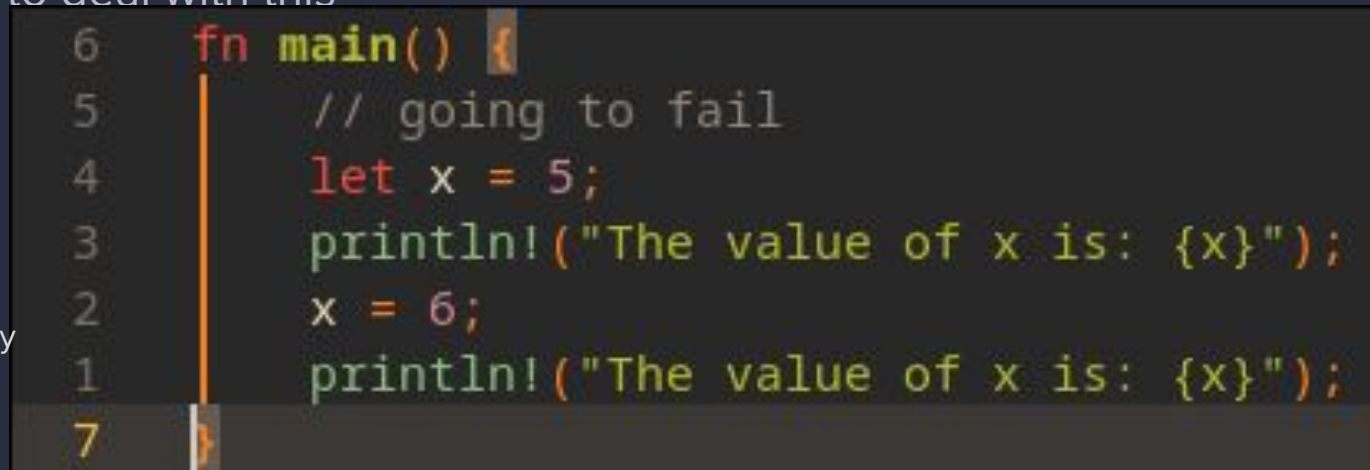
Variables and Mutability

When you try to run the code here, you will run into an error similar to the one show below.

Variables in Rust are immutable by default.

Use the mut keyword to deal with this

rust-workshop/var-and-mutability



Resolve Mutability Issue

```rust
fn main() {
    // Resolved using mut keyword
    let mut x = 5;
    println!("The value of x is: {x}");
    x = 6;
    println!("The value of x is: {x}");
}
```

```rust
fn main() {
    // Resolved using shadowing
    let x = 5;
    println!("The value of x is: {x}");
    let x = 6;
    println!("The value of x is: {x}");
}
```

```
noah@noah-MS-7E13:~/projects/rust-workshop/rust_basics/var-and-mutability$ cargo run
   Compiling var-and-mutability v0.1.0 (/home/noah/projects/rust-workshop/rust_basics/var-and-mutability)
    Finished dev [unoptimized + debuginfo] target(s) in 0.45s
     Running `target/debug/var-and-mutability`
The value of x is: 5
The value of x is: 6
```

rust-workshop/var-and-mutability

Integers

Integer Size Guide

Length	Signed	Unsigned
8-bit	`i8`	`u8`
16-bit	`i16`	`u16`
32-bit	`i32`	`u32`
64-bit	`i64`	`u64`
128-bit	`i128`	`u128`
architecture dependent	`isize`	`usize`

Example of integers

```rust
fn main() {
    // create a variable
    // we are explicitly seting x to i32 type
    let x: i32 = -23;
    let y: u32 = 654;
    // 64 bit unsigned integer
    let z: u64 = 12034981927589017;
}
```

rust-workshop/integers

Some Math with our Integers

Add, Subtract, Multiply, Divide, and Modulo

- We are going to some math with integers in rust.
- Lets see what happens when we run the following code.

```rust
fn main() {
    let x = 6;
    let y = 5;

    let mut z = x + y; // adding
    println!("{z}");

    z = x - y; // subtraction
    println!("{z}");

    z = x * y; // multiplication
    println!("{z}");

    z = x / y; // What happens when we divide integers?
    println!("{z}"); // we get an integer

    z = x % y; // module or remainder division
    println!("{z}");
}
```

rust-workshop/math

Floats

Floats

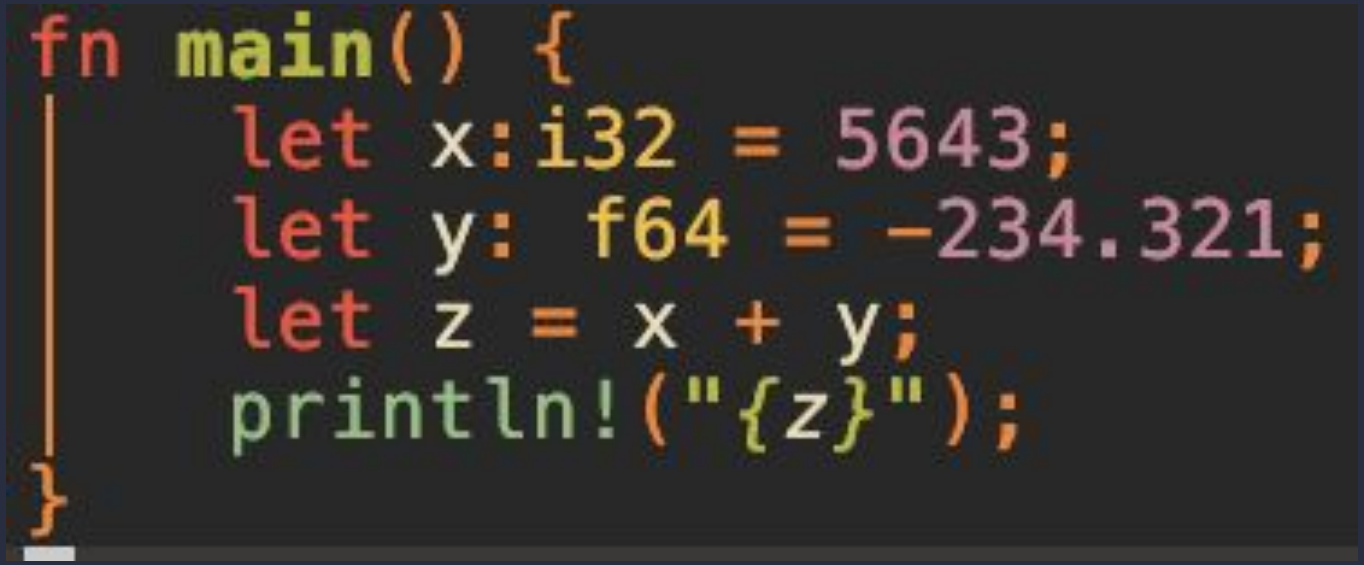
Floats are numbers that can have decimals. Similarly names to integers. i64 is a 64 bit integer. f64 is a 64 bit floating point number. You can also do math with floating point numbers

```
fn main() {
    let y: f64 = 5.2341543;
    let x = -431.23;
    let z = x + y;
    println!("{z}");
}
```

rust-workshop/floats

Let's add Floats and Integers!

Hey guys! I really want to add this float x and this integer y. I am absolutely sure that there will be nothing wrong here. Yup!



rust-workshop/floats_and_ints

Wait... What???

Yup. You can not add integers and floats together. Make sure to check your types with different operations.

```
error[E0277]: cannot add `f64` to `i32`
 --> src/main.rs:4:15
  |
4 |     let z = x + y;
  |               ^ no implementation for `i32 + f64`
  |
  = help: the trait `Add<f64>` is not implemented for `i32`
  = help: the following other types implement trait `Add<Rhs>`:
            `&i32` implements `Add<i32>`
            `&i32` implements `Add`
            `i32` implements `Add<&i32>`
            `i32` implements `Add`

For more information about this error, try `rustc --explain E0277`.
error: could not compile `floats_and_ints` (bin "floats_and_ints") due to 1 previous error
```

rust-workshop/hello-world

Compound Types

Tuples

Compound types group multiple values into one type.

Tuple Types a set number of values that can be any type.

Can have an integer with a float in a tuple.

```rust
fn main() {
    // Tuple type
    let tup: (i32, f64, u8) = (500, 6.4, 1);
    println!("The tuple's values: {}, {}, {}", tup.0, tup.1, tup.2);

    // Tuple unpacking
    let (x, y, z) = tup;
    println!("The value of x is: {}", x);
    println!("The value of y is: {}", y);
    println!("The value of z is: {}", z);
}
```

rust-workshop/tuple-type

Array

Array is a compound type that groups together data of a the same type.

You can have an integer array that contains integers and only integers.

```rust
fn main() {
    // Integer array
    let array: [i32; 5] = [1, 2, 3, 4, 5];
    println!("The first element in array is {}", array[0]);
    // array needs special notation to print to std out
    println!("{:?}", array);
}
```

rust-workshop/array-type

Functions

Your First Function

Here is an example of defining and using a function within Rust.

Think of a function as a way to reuse code

```
6  fn main() {
5      my_first_function();
4  }
3
2  fn my_first_function() {
1      println!("Hello. I am your function. YIPPIE!!!");
7  }
```

rust-workshop/functions

Function with Parameters

```
fn function_with_parameters(x: u32) {
    println!("The number that you put in is {x}")
}
```

```
The number that you put in is 27
noah@noah-chromeback:~/sd/rust-workshop/rust_basics/functions$
```

rust-workshop/functions

Expressions and Statements

Expression

Expressions evaluate to a resultant value.

Does not end with ;

```
fn main() {
    let z = 5;

    let x = {
        let y = 5;
        y + 1
    };
    println!("{x}");
}
```

rust-workshop/expression

Statement

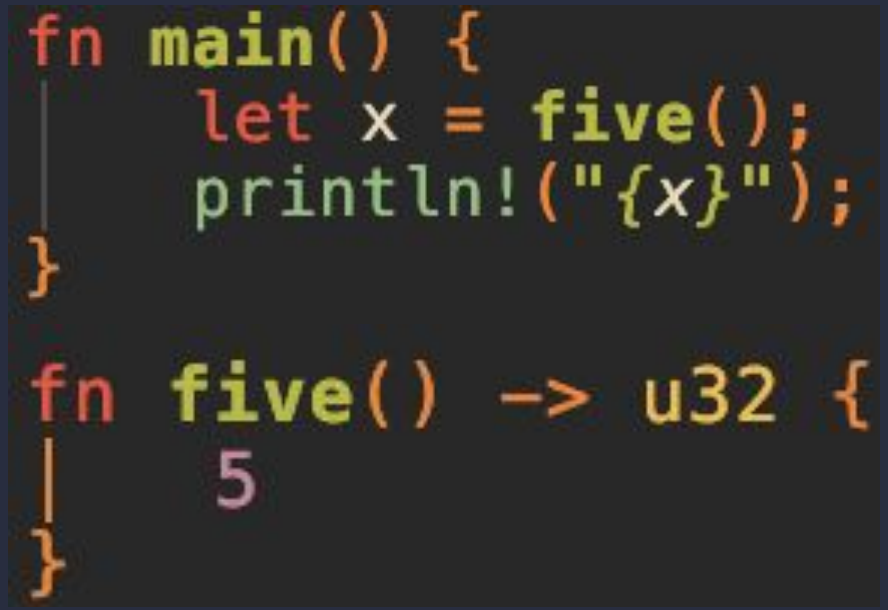
Statements are instructions that perform some action and do not return a value.

End with ;

```
fn main() {
    let z = 5;

    let x = {
        let y = 5;
        y + 1
    };
    println!("{x}");
}
```

Applying Expressions to Functions



rust-workshop/functions

Putting it All Together

Lets create a function that returns the sum of two numbers.

```rust
fn main() {
    let x: u32 = 5;
    let y: u32 = 10;
    let z: u32 = sum(x, y);
    println!("{z}");
}

fn sum(x: u32, y: u32) -> u32 {
    // or you can use return x + y to forcibly return a value
    x + y
}
```

rust-workshop/functions

Control Flow

Booleans

A boolean is a data type that equates to true or false



rust-workshop/control-flow

if statement (conditional)

Conditionals are how we are able to control our program and make it smart.

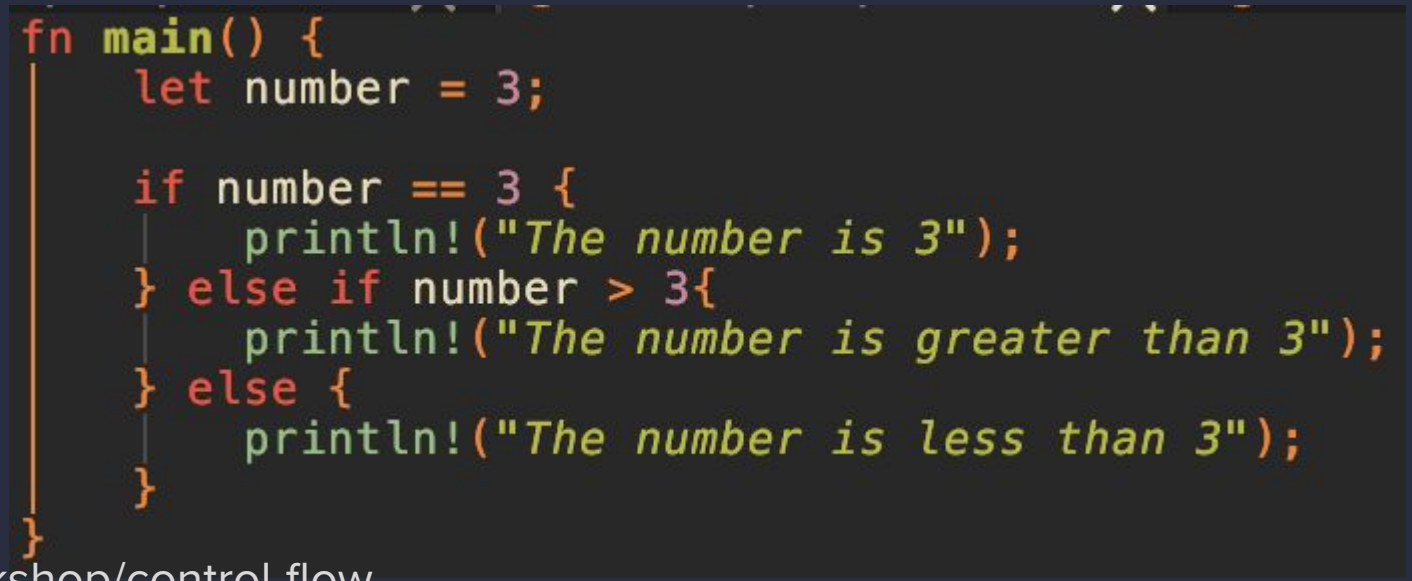
If check a boolean value if it is true or false



rust-workshop/control-flow

Else and Else If

If you need to handle other cases, you can use else if and else to handle other cases.



```rust
fn main() {
    let number = 3;

    if number == 3 {
        println!("The number is 3");
    } else if number > 3{
        println!("The number is greater than 3");
    } else {
        println!("The number is less than 3");
    }
}
```

rust-workshop/control-flow

Conditional Expressions????

Yup, conditionals can also be expressions

```
fn main() {
    let number = 3;

    let my_number = if number > 3 {
        "greater than three"
    } else {
        "less than or equal to three"
    };

    println!("{my_number}");
}
```

rust-workshop/control-flow

Loops

Loops are how we can iterate our code

Loop Loops

The loop loop will iterate forever.

Press Ctrl + c to stop the program (btw).

```rust
use std::{thread, time};
fn main() {
    let one_second: Duration = time::Duration::from_secs(1);
    loop {
        println!("Hey there");
        thread::sleep(dur: one_second);
    }
}
```

rust-workshop/loop-loop

While Loops

The while loop will until some condition returns false.

```
fn main() {
    let a = [10, 20, 30, 40, 50];
    let mut index = 0;

    while index < 5 {
        println!("the value is: {}", a[index]);

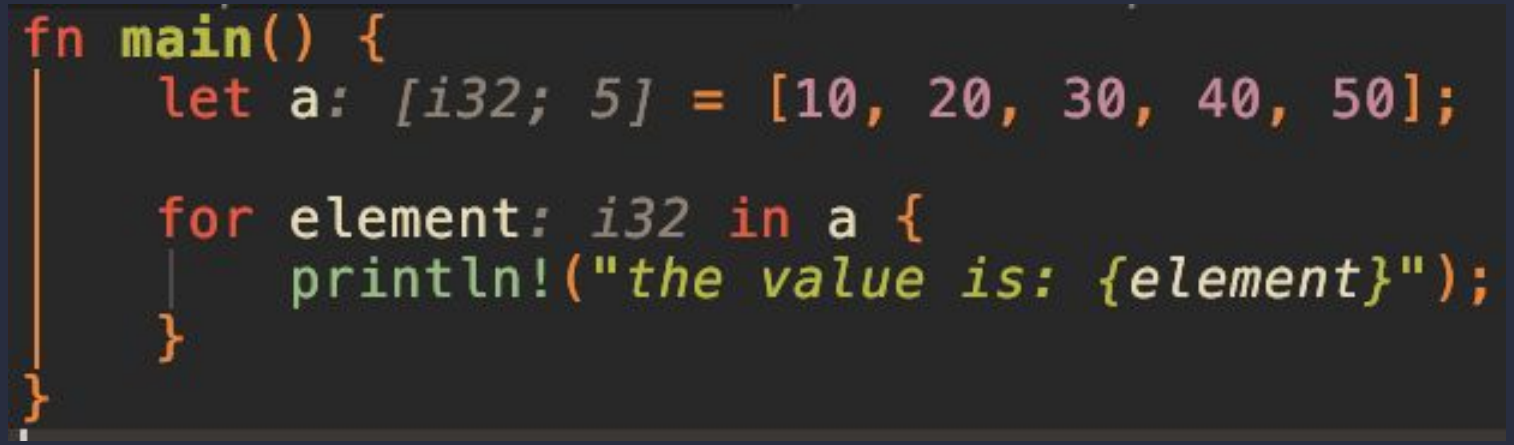
        index += 1;
    }
}
```

rust-workshop/while-loop

For Loops

The for loop will go through the items in array like structures.

Will end eventually.



rust-workshop/for-loop

Comparison between loops

Loop loops

We do not want the loop to end.

If you create a server, you want that server to run until the administrator manually stops the server.

While loops

We want the loop to end, but do not know when it will end.

We want to add a number until it is greater than 100

It may never reach that.

For loops

We want the loop to end and know when it will end.

We want to go through an array of integers.

End when it reaches the end of the array.

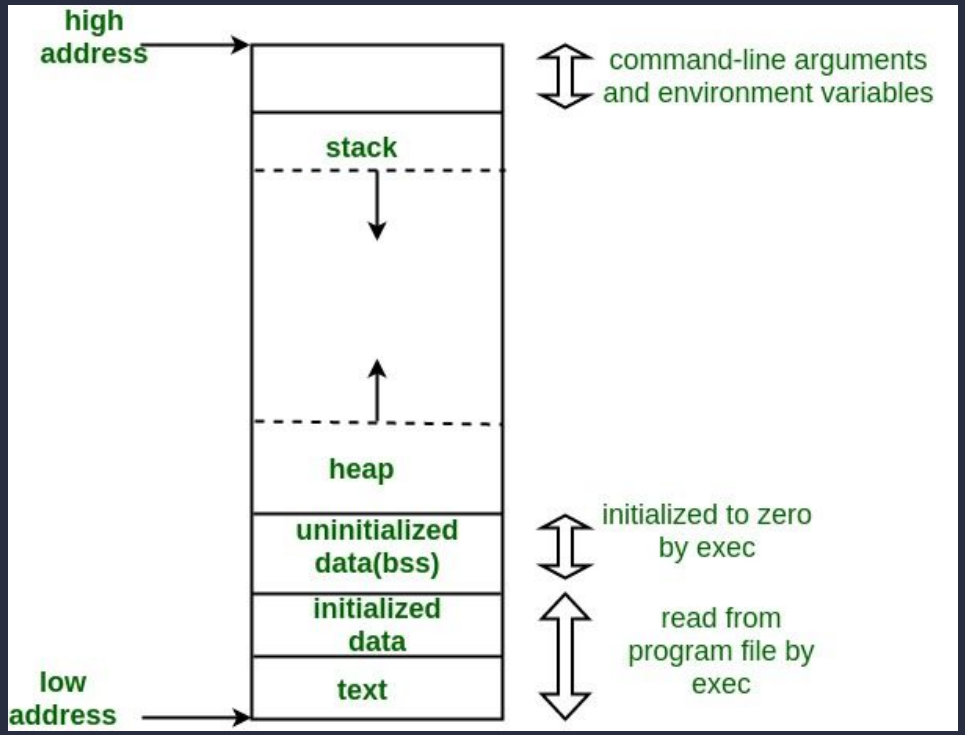
Ownership

What is ownership?

Ownership is the set of rules that rust uses to govern memory management.

In rust, you will manage the stack and the heap when using your memory.

Think of the stack as a more organized area of memory and heap a less organized area.



Stack Analogy

You are working in an office space, and need to fill out papers like a computer needs to run programs.

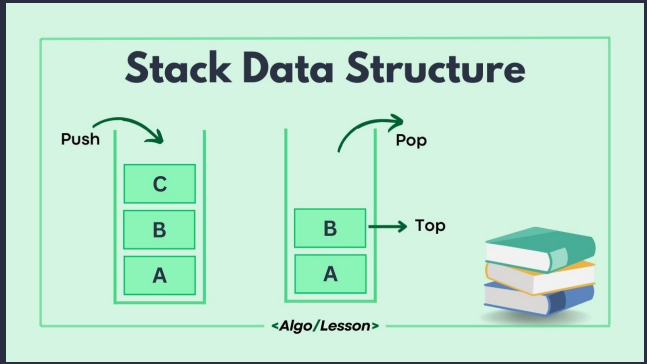
On your desk, you start filling out forms. You put the form in front of you.

As you fill out your first form, you find that one field requires you to fill out another form. So you take the next form, put it on top of the other and start filling it out.

When you are done filling out the second form, you remove it from your stack of forms and continue filling out the first form.

This is similar to how a computer pushes and pops memory on a stack to execute its program.





Heap Analogy

To continue with our analogy. Imagine that you need the space for a form. You do not know when you will need to finish filling it out, but know that you need it somewhere you can access.



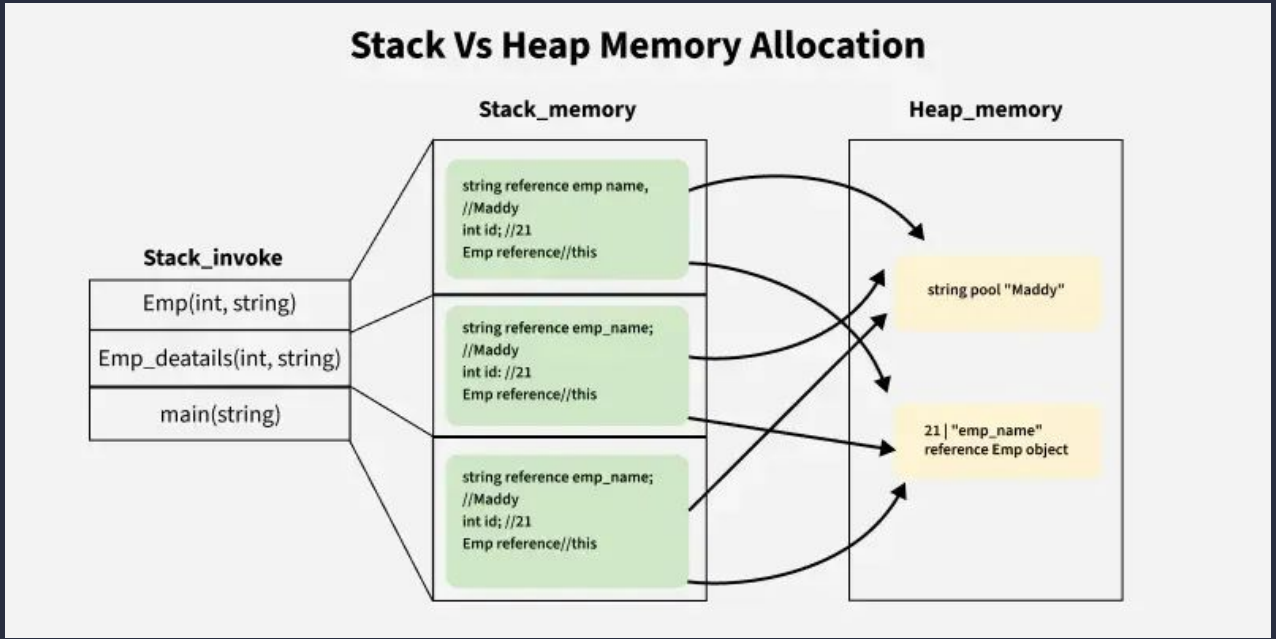
Imagine that you have a separate desk with space. When you need to pull out another form, you put it on the table. Eventually, you will have to look for space for these forms when you put a new one on.

Similarly, a computer will find space in its heap memory to allocate and keep track of where that is.

Stack vs. Heap

The computer has immediate access to memory in the stack. Not true with heap.

To solve this, we can store references or pointers of heap memory in stack memory.



The Three Rules of Ownership

1. Each value in Rust has an owner.
2. There can only be one owner at a time.
3. When the owner goes out of scope, the value will be dropped.

```rust
fn main() {
    {
        // create variable x in this scope
        // x owns the string "My value"
        let x = String::from("My value");
        // saying that y owns the value of x
        let y = x;
        // Because x does not own the value
        // It is invalid
        println!("{x}");
    }
    // y is out of scope, you can not reference it after
    println!("{y}");
}
```

rust-workshop/ownership

Fixed code

```
// Fixed code
let y = {
    let x = String::from("Hello world");
    // clone the data from x
    // y owns that clone
    let y = x.clone();
    println!("{x}");
    // y is the expression for this block
    y
};
println!("{y}");
```

rust-workshop/ownership

References

References

When you call a function, when you pass a parameter, you are also passing ownership to that value.

Instead, you can pass a reference parameter, which will refer to a value rather than owning it.

```rust
fn main() {
    // create a string
    let s1 = String::from("hello");

    // pass a reference to s1
    let len = calculate_length(&s1);

    // s1 is still valid after passing it's reference
    println!("The length of '{s1}' is {len}.");
}

fn calculate_length(s: &String) -> usize {
    // s goes out of scope
    // but because it is a reference,
    // that original data is not deallocated
    s.len()
}
```

rust-workshop/references

Mutable References

You can also modify the original data with a mutable reference.

NOTE: The original value must be mutable.

```
fn change(some_string: &mut String) {
    some_string.push_str(", world");
}

let mut s2 = String::from("hello");

change(&mut s2);

println!("{s2}")
```

rust-workshop/references

Borrowing

Borrowing

Generally, when we reference data, read and write to that data, this is called borrowing.

That reference does not own the data and will not deallocate the data when it becomes invalid.

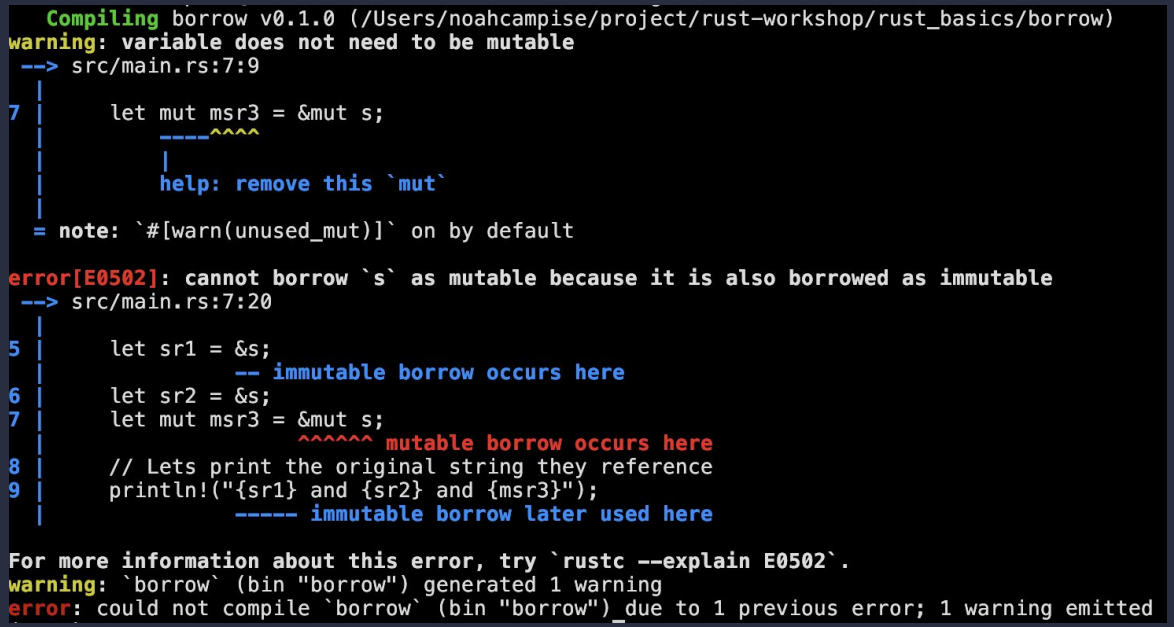
There are rules to borrowing as well.

```rust
fn main() {
    // create a variable x
    let mut s = String::from("Hello World");
    // create 2 references and 1 mutable reference
    let sr1 = &s;
    let sr2 = &s;
    let mut msr3 = &mut s;
    // Lets print the original string they reference
    println!("{sr1} and {sr2} and {msr3}");
}
```

rust-workshop/borrow

Uh Oh ... More Rules

When borrowing in rust, you can either have any number of immutable references or one and only one mutable reference at any line of code.



rust-workshop/hello-world

Fixed

Here is how we can resolve this issue and still have our references.

```
let mut s = String::from("Hello World");
// create 2 references
let sr1 = &s;
let sr2 = &s;
// Lets print the original string they reference
println!("{sr1} and {sr2}");

// create a mutable reference
let mut msr3 = &mut s;
// sr1 and sr2 are never used again, we are fine
println!("{msr3}");
```

rust-workshop/borrow

Last Thing, Must Borrow Valid Data

When borrowing data, that data must be valid. Otherwise, it would be a dangling reference.

```
fn dangling() -> &String {
    let str = String::from("Perfectly fine string");

    &str
}
```

rust-workshop/borrow

Slices

Slices

You can create a reference to a section of a string or array using a slice. You can also pass this slice as a parameter:

```rust
fn main() {
    let a = [1, 2, 3, 4, 5, 6];
    let slice = &a[1..3];
    println!("{:?}", slice);

    let owned_str = String::from("Hello world");
    let reference = first_word(&owned_str);
    println!("{reference}");
}

// takes a string slice as a parameter
fn first_word(s: &str) -> &str {
    // returns a slice of bytes
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        // checks if there is a space
        if item == b' ' {
            // returns the reference early
            return &s[0..i];
        }
    }

    // references the original data, which is still valid outside
    // of the function
    &s[..]
}
```

rust-workshop/slices

Structs

What is a Struct?

A struct is a grouping of related data into one type. Similar to tuple, but uses fields to access parts of data.

Similar to classes or objects in other programming languages.

```
struct User {
    // template for a User struct
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,
}

fn main() {
    let user1 = User {
        // fill in values for that type
        active: true,
        username: String::from("someusername123"),
        email: String::from("someone@example.com"),
        sign_in_count: 1,
    };
    println!("User1's username is {}", user1.username);
}
```

rust-workshop/structs

Mutating a Struct

You can change values of a struct by setting the value of a struct's field.

NOTE: You can not choose which fields can and can not be mutated. The whole struct is either mutable or not.

```rust
fn main() {
    let mut user1 = User {
        // fill in values for that type
        active: true,
        username: String::from("someusername123"),
        email: String::from("someone@example.com"),
        sign_in_count: 1,
    };
    println!("User1's username is {}", user1.username);
    user1.active = false;
    if user1.active {
        println!("User1 is active");
    } else {
        println!("User1 is inactive");
    }
}
```

rust-workshop/structs

Methods

Implementing Methods

Methods are functions defined within the context of a struct.

Self refers to Rectangle the template.

self refers to this specific instance of a rectangle

rust-workshop/methods

```rust
// used to allow our struct to be printed
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

// impl block is where we will implement methods
impl Rectangle {
    // constructor like method, returns an instance
    // of specified using Self
    fn new(width: u32, height: u32) -> Self {
        Rectangle { width, height }
    }

    // returns area of Rectangle
    // reference to self, so function does not own self
    fn area(&self) -> u32 {
        // unlike c or c++, we can use . instead of ->
        self.width * self.height
    }
}
```

Using our Methods

```
fn main() {
    let rect: Rectangle = Rectangle::new(width: 5, height: 5);
    println!("{rect:?}");
    let area: u32 = rect.area();
    println!("The area is {area}");
}
```

rust-workshop/methods

Enumerations

What is an Enumeration?

Enumerations give you a way to say a value is one of several possible values.

Lets see how one could use and enumeration

```rust
#[derive(Debug)]
enum IpAddrKind {
    // IP address enumeration
    V4,
    V6,
}

// We can accept the enumeration, even if it is a different type
fn search_ip(ip: &IpAddrKind) {
    println!("{ip:?}");
}

fn main() {
    let ip4 = IpAddrKind::V4;
    search_ip(&ip4);
    let ip6 = IpAddrKind::V6;
    search_ip(&ip6);
}
```

rust-workshop/enumerations

Unique Values

Enumeration variants can have their own unique values.

```
// each variant can have its own, unique data
#[derive(Debug)]
enum Message {
    Quit,                       // no data
    Move { x: i32, y: i32 },    // struct like fields
    Write(String),              // single string
    ChangeColor(i32, i32, i32), // includes 3 i32
}
```

rust-workshop/enumerations

Enumeration Methods

You can create methods for enumerations the same way you would for structs.



rust-workshop/enumerations

Option Enumeration

As part of the Standard Library, there are Option Enumerations.

It is used for handling unexpected behavior.

```
enum Option<T> {
    None,
    Some(T),
}
```

```
// Create an option
let option_string = Option::Some(String::from("option"));
// Either retrieve the value if it is Some
// of if there is a None, end the program
let string = option_string.unwrap();
```

rust-workshop/enumerations

Matching

Match Blocks

If you come from c or c++, match blocks are similar to switch statements

```rust
fn char_to_i32(c: char) -> i32 {
    // exhausts all possible values of param c
    match c {
        // can have a code block
        'a' => {
            let x: i32 = 1;
            x
        }
        // or can just be an expression followed by a comma
        'b' => 2,
        _ => 3, // _ is the default value
    }
}

fn main() {
    let c1: char = 'a';
    let i1: i32 = char_to_i32(c1);
    println!("{i1}");
}
```

rust-workshop/matching

Pattern Matching Enumerations

Pattern matching is the mechanism that Rust Uses to handle variants of an enumerations.

For a given enumeration, we know that it is one of several variantes.

Pattern Matching allows us to define functionality for any one variant.

```
#[derive(Debug)]
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(State),
}

// Derive the Clone trait
// Allows us to clone the state enumeration
#[derive(Debug, Clone)]
enum State {
    Pennsylvania,
    Florida,
    Washington,
    Virginia,
    NewYork,
}
```

rust-workshop/matching

Pattern Matching Enumerations Cont.

```rust
impl Coin {
    fn coin_to_value(&self) -> u32 {
        match self {
            // Remember, Self is saying a Coin in general
            // Not this particalar coin instance self
            Self::Penny => 1,       // for Penny variant
            Self::Nickel => 5,      // for Nickel variant
            Self::Dime => 10,       // for Dime variant
            Self::Quarter(_) => 25, // for Quarter variant
        }
    }

    fn quarter_state(&self) -> Option<State> {
        match self {
            // pattern matching used to borrow the variants data
            Coin::Quarter(state) => Some(state),
            _ => None,
        }
    }
}
```

rust-workshop/matching

Matching in Action … With if let Conditional

The if let block will match to one variant of an enumeration, otherwise run the else block.

```rust
fn main() {
    let c1 = 'a';
    let i1 = char_to_i32(c1);
    println!("{i1}");

    // our functions in action
    let my_coin = Coin::Quarter(State::NewYork);
    let value = my_coin.coin_to_value();
    println!("{value}");

    let my_state: Option<State> = my_coin.quarter_state();
    // if let Some(state) will pattern match to the
    // variant of my_state
    if let Some(state) = my_state {
        println!("The state of my quarter is {state:?}");
    } else {
        println!("The coin was not a quarter");
    }
}
```

rust-workshop/matching

Stuff to Check Outside Workshop

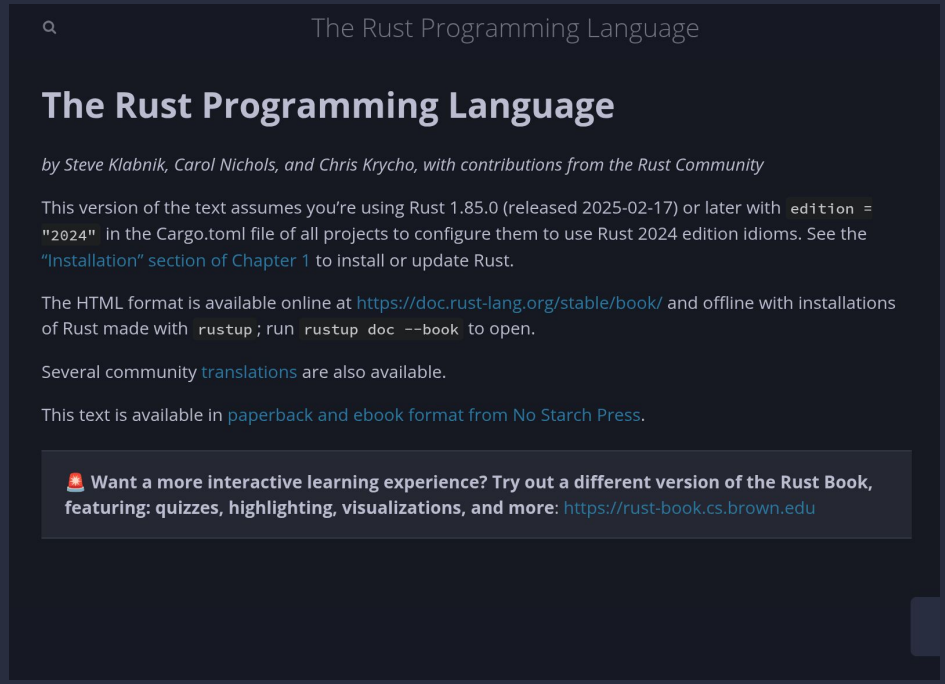
DOCUMENTATION!!!

Rust has extensive documentation available online.

Rust Programming Book available online for FREE!!!

[Rust Book Link]





What should you take away?

- This is an introduction to Rust Programming.
- You should be understand design patterns in Rust API's and Libraries.
- You should be able to understand how to build more complex applications.

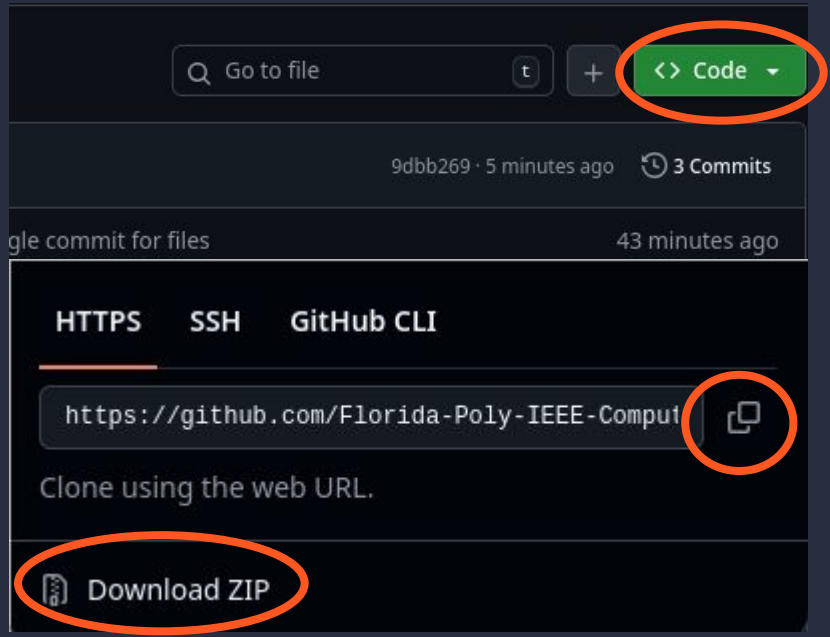


Thank You!
Please Join us for Part II!

Calculator Project

Get the Repository

- Highly suggested that you have git installed on your machine to go between the different branches.
- Rust Workshop Repository
- Either download the zip or run the command “git clone <repository url>.



Create the cargo project folder

Run the following in your terminal.

“cargo new calculator_project”

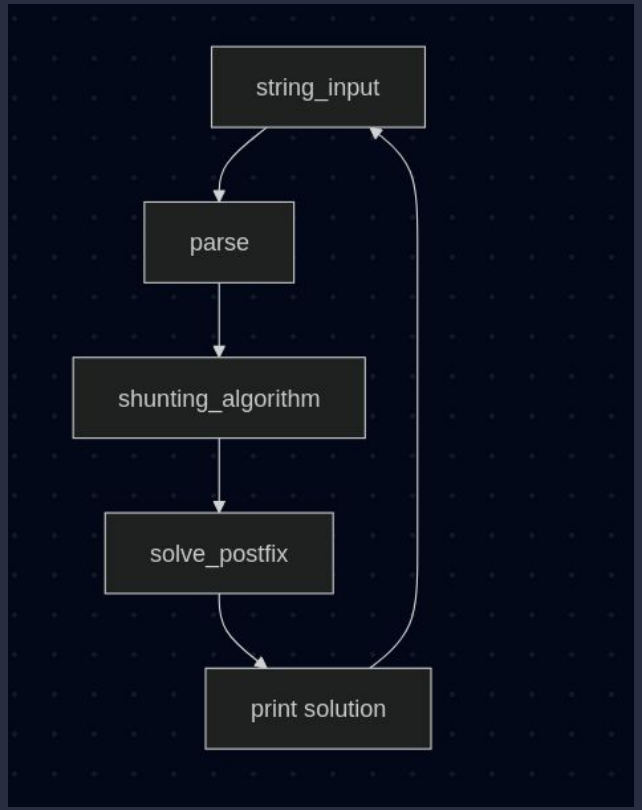
This will create the directory that your put your source code into and run.,

To run, cd into that directory and run “cargo run”

```
noah@noah-MS-7E13:~/projects$ cargo new calculator_project
    Creating binary (application) `calculator_project` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
noah@noah-MS-7E13:~/projects$ cd calculator_project/
noah@noah-MS-7E13:~/projects/calculator_project$ cargo run
   Compiling calculator_project v0.1.0 (/home/noah/projects/calculator_project)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.11s
     Running `target/debug/calculator_project`
Hello, world!
```

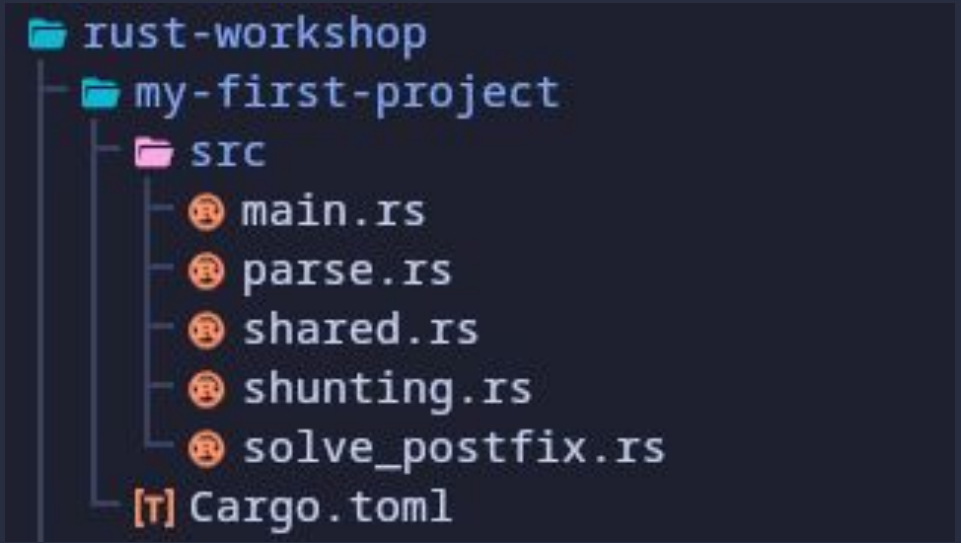
Runtime Breakdown

- Calculator app is going to be a loop.
- At each iteration, read from stdin, parse infix input, map infix to postfix, solve postfix, and restart.
- As for why, this is what a typical calculator will use to solve an equation.



Project File Breakdown

- 5 rust files total
- main.rs - Main function that is run.
- parse.rs - Parse string input to enumerations.
- shunting.rs - Converts infix enumerations to postfix enumeration.
- solve_postfix.rs - Solves postfix enumerations, returning the solution.
- shared.rs - Shared structs and enumerations.



Why parse the input?

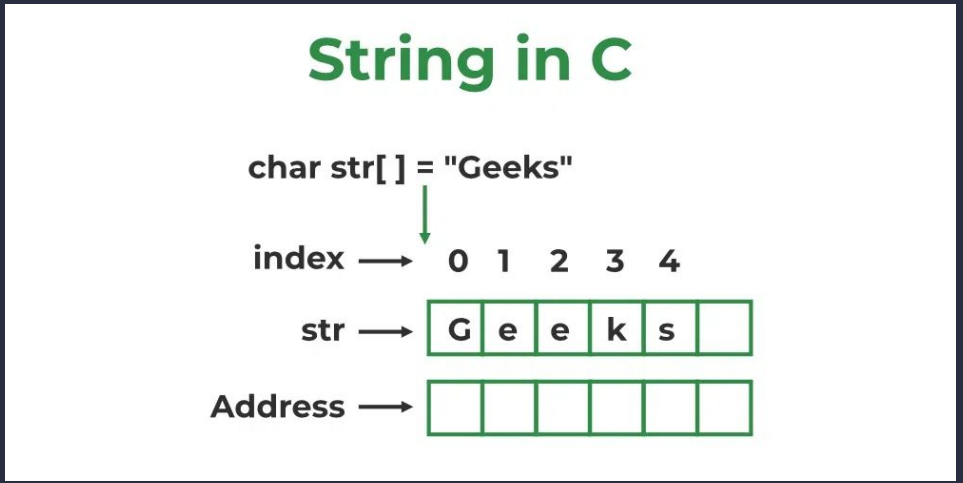
Understanding what the input is.

The user is inputting a string, which is just an array of characters.

There is no inherent meaning to that string.

How can our program know how to distinguish a floating point number from a whole number?

In order for our program to recognize what exactly the users means when they input their value, we need some way to parse that value.



What do we need to parse input

We need some way to tokenize (internalize a string input into an array of tokens) the string input so that our application knows the difference in the expression 534.1 + 4 * 32.1:

- 534.1
- +
- 4
- *
- 32.1

Creating our Tokens with ENUMERATIONS

In order to parse, we need to tokenize the possible values that someone can input.

An easy way to do this in Rust is to use enumerations.

Think of enumerations as a type that can be one and only one subtype.

```rust
// Enumeration for the different token types that we are expecting
// to push on to the stack
#[derive(Debug, Clone)]
pub enum Token {
    Whole(f64),
    Floating(u64, f64),
    Operator(Operator),
}

#[derive(Debug, Clone)]
pub enum Operator {
    Add,
    Sub,
    Mul,
    Div,
    OpenParen,
    CloseParen,
}
```

Creating our array of tokens with VECTORS

- The size of an array can not be changed.
- So we will use the vector.
- Think of a vector as an array that we can increase or decrease in size throughout our application.
- Here we are saying that the parse function returns a vector of Tokens.

```
// Create an algorithm that takes an
// infix string, and parse into a Queue of tokens
pub fn parse(infix_str: &str) -> Vec<Token> {}
```

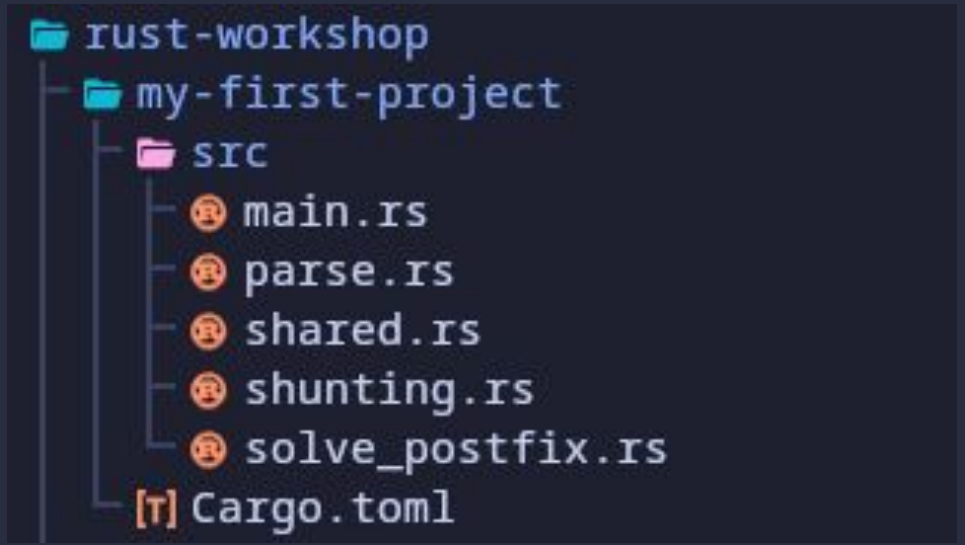
Live Programming Parse Functionality

Add code to parse.rs, shared.rs, main.rs.

Add enumerations to parse.rs.

Add parse functionality to parse.rs.

Add modules to main.rs.



Infix to Postfix: Shunting Yard Algorithm

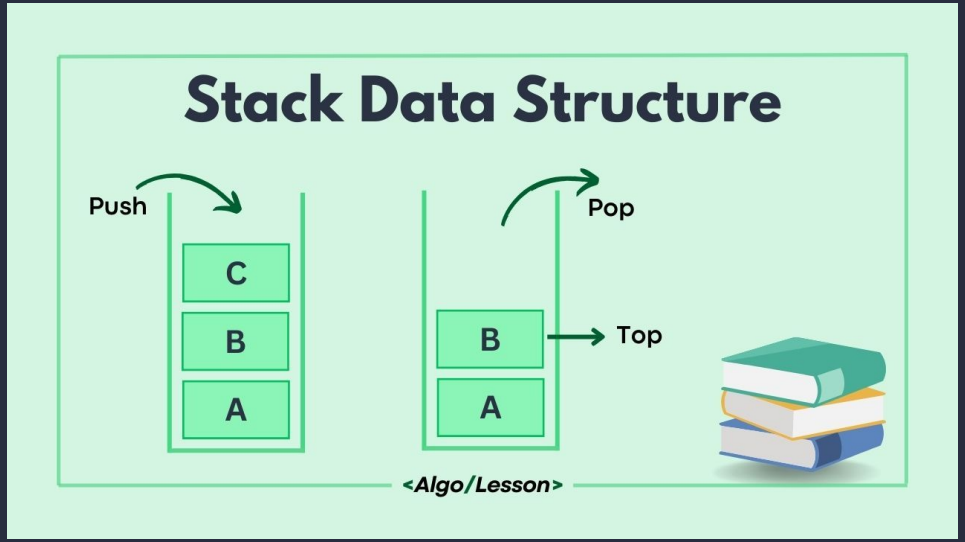
What is infix vs postfix?

Infix Expression	Postfix Expression
1 x 2 + 3 / 4	1 2 x 3 4 / +

Why convert infix to postfix?

Two Reasons

1. Removes parentheses from the equations.
2. You can use a stack to do your equations.



Shunting Yard Algorithm

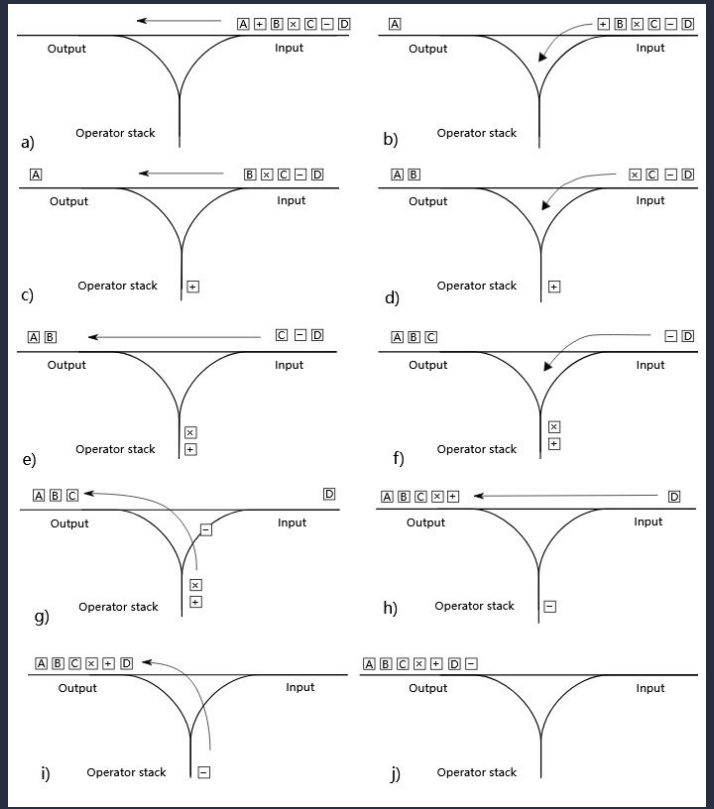
Go through the tokens in the infix equation and add them to a queue.

When given an operator, push operators on to a stack while going through the tokens.

Pop items off of the stack depending on parenthesis and order of operations

At the end, pop all of the items off of the stack into the queue.

If you want to look at a more detailed description of shunting yard algorithm: [wikipedia]



How to we create this queue?

Structs and Type Aliases

We will use a struct as a way to create our output queue from the Shunting Yard Algorithm.

Structs are a type that acts as an aggregate of subtypes, which are accessed using their field name.

Use a type alias to create an operator stack.

Think of a type alias renaming another type.

```rust
// OutputQueue used in the Shunting Yard Algorithm
#[derive(Debug, Clone)]
pub struct OutputQueue {
    data: VecDeque<Token>,
}

// OperatorStack is a type alias for a Vector of Tokens
pub type OperatorStack = Vec<Operator>;
```

How do we add order of operations?

Implementation blocks and traits.

Implementation blocks (impl blocks) are blocks of code that implement methods for structs and enumerations.

Traits are a way to create shared behavior of types. For our case, just know that this is how we can get our Operator enumerations to be compared using ==, >=, <=, <, >

```rust
impl Operator {
    // Maps operators to u8 for comparison
    // Used to compare order of operation
    fn operator_order(&self) -> u8 {
        match self {
            Operator::Add => 1,
            Operator::Sub => 1,
            Operator::Mul => 2,
            Operator::Div => 2,
            Operator::CloseParen => 4,
            Operator::OpenParen => 4,
        }
    }
}

// Trait used to compare operators with == and !=
impl PartialEq for Operator {
    fn eq(&self, other: &Self) -> bool {
        self.operator_order() == other.operator_order()
    }
}
```

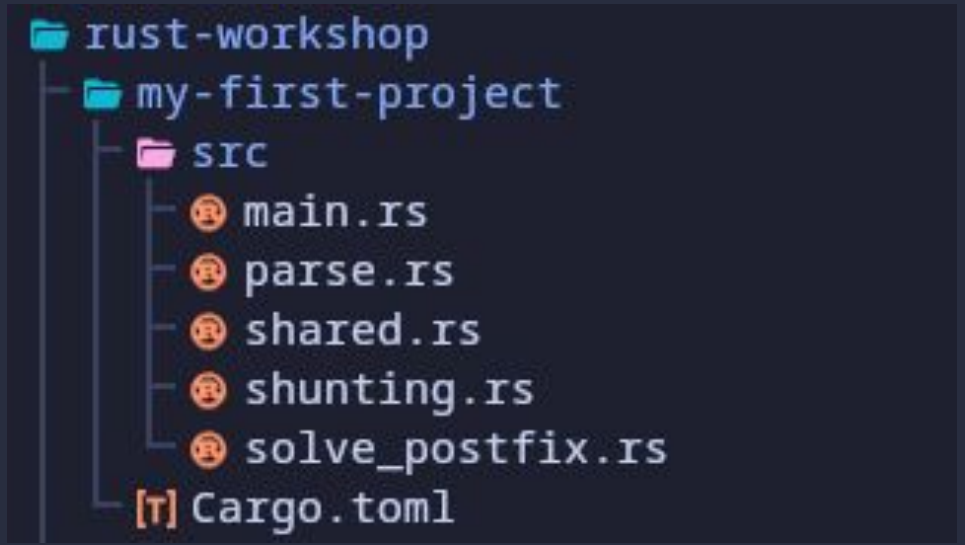
Live Programming Shunting Functionality

Add code to parse.rs, shared.rs, main.rs.

Add queue struct to parse.rs.

Add shunting yard algorithm to shunting.rs.

Add modules to main.rs.



Solving Postfix

Final part of the project.

We will loop through the output_queue from the shunting yard algorithm created previously.

Throughout, we will use a stack in order to deal with the operations that are in postfix.

```rust
pub fn solve_postfix(output_queue: OutputQueue) -> f64 {
    // To solve in postfix, we will need a stack and logic to handle the difference cases
    let mut stack: Vec<f64> = Vec::new();
    // loop through each token in the queue
    for i: Token in output_queue {
        match i {
            Token::Whole(val: f64) => stack.push(val),
            Token::Floating(_, val: f64) => stack.push(val),
            Token::Operator(operator_type: Operator) => {
                // The operation should go: x operation y
                let x: f64 = stack.pop().expect(msg: "Stack should not be empty");
                let y: f64 = stack.pop().expect(msg: "Stack should not be empty");
                match operator_type {
                    Operator::Add => stack.push(add_floats(x, y)),
                    Operator::Sub => stack.push(substract_floats(x, y)),
                    Operator::Mul => stack.push(multiply_floats(x, y)),
                    Operator::Div => stack.push(divide_floats(x, y)),
                    _ => panic!("Unexpected operator"),
                }
            }
        }
    }
    stack Vec<f64>
        .pop() Option<f64>
        .expect(msg: "Calculation finished, should have succeeded")
} fn solve_postfix
```

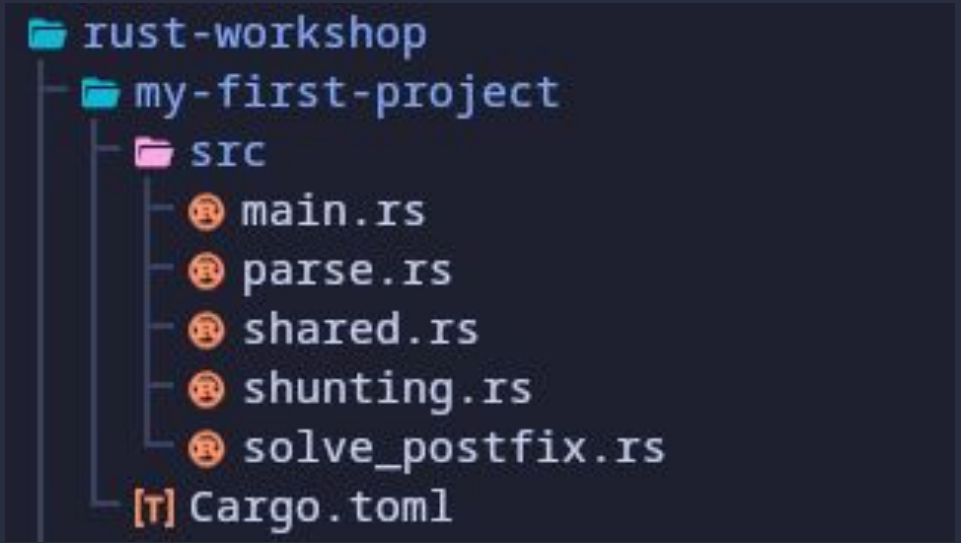
Live Programming Postfix Functionality

Add code to solve_postfix.rs and main.rs.

Add solve_postfix function to solve_postfix.rs.

Combine everything into main.rs.

Finish the project :)



Final Result

```
noah@noah-MS-7E13:~/projects/rust-workshop/my-first-project$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
     Running `target/debug/my-first-project`
Welcome to the Rust Workshop Calculator Command Line Tool!
Please input a value:
14 + 32.3 - 32 * 2 + (3 - 1)
-15.700000000000003
Please input a value:
14 + 32.3
46.3
Please input a value:
14 + 32.3 - 32
```